In this chapter, you'll see:

- How to change RSpec's behavior at the command line
- How to customize RSpec's output
- Where to save your commonly used command-line options
- How to configure RSpec in code
- Which configuration options will be useful on your projects

CHAPTER 9

Configuring RSpec

As you've worked through the exercises in this book, you've often changed RSpec's behavior to make it a better tool for your needs. Here are just a few of the things you've customized RSpec to do for you:

- Set up and tear down a test database, but only for the examples that require one
- Report *every* failing expectation in an example, not just the first one
- Run just the examples that you're focusing on at the moment

In this chapter, we're going to connect the dots between all these individual customizations. By the end, you'll know how to configure much more than a few individual settings. You'll have a solid overview of RSpec's configuration system. Instead of having a general-purpose tool, you'll have a testing environment tailor-made to fit your workflow.

You can configure RSpec in two basic ways:

- *An RSpec.configure block*: Provides access to *all* configuration options; since this block lives in your code, you'll typically use it to make permanent changes
- *Command-line options*: Provides access to *some* configuration options, typically one-off settings that will affect a specific `rspec` run

We're going to take a closer look at both of these categories, beginning with command-line options.

Command-Line Configuration

To see all the available command-line options, run `rspec --help`. You'll get a pretty big list of settings, some of which you've already used in this book.